

Start coding or generate with AI.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Double-click (or enter) to edit

```
from google.colab import files
uploaded=files.upload()
print(uploaded)
```

```
data=pd.read_csv("diabetes.csv.csv")
data.head()
```

Choose Files diabetes.csv.csv

- diabetes.csv.csv(text/csv) - 23875 bytes, last modified: 9/1/2025 - 100% done

Saving diabetes.csv.csv to diabetes.csv (1).csv

{'diabetes.csv (1).csv': b'Pregnancies,Glucose,BloodPressure,SkinThickness,Insulin,BMI,DiabetesPedigreeFunction,Age,Outcome\r\n6,148,72,35,0,33.6,0.627,50,1\r\n1,85,66,29,0,26.6,0.351,31,0\r\n2,183,64,0,0,23.3,0.672,32,1\r\n3,89,66,23,94,28.1,0.167,21,0\r\n4,137,40,35,168,43.1,2.288,33,1'}

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome	
0	6	148	72	35	0	33.6	0.627	50	1	
1	1	85	66	29	0	26.6	0.351	31	0	
2	8	183	64	0	0	23.3	0.672	32	1	
3	1	89	66	23	94	28.1	0.167	21	0	
4	0	137	40	35	168	43.1	2.288	33	1	

Next steps:

Generate code with data

☒ View recommended plots

New interactive sheet

EDA(Exploratory Data Analysis)

```
data["Pregnancies"].value_counts()
```



count	
Pregnancies	
1	135
0	111
2	103
3	75
4	68
5	57
6	50
7	45
8	38
9	28
10	24
11	11
13	10
12	9
14	2
17	1
15	1

dtype: int64

data.columns



```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
      'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='object')
```

len(data.columns)



```
9
```

data.info()



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Pregnancies           768 non-null   int64
1   Glucose               768 non-null   int64
2   BloodPressure         768 non-null   int64
3   SkinThickness         768 non-null   int64
4   Insulin               768 non-null   int64
5   BMI                  768 non-null   float64
6   DiabetesPedigreeFunction 768 non-null   float64
7   Age                  768 non-null   int64
8   Outcome              768 non-null   int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

data.describe()



	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

```
data.isnull().sum()
```



	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

```
data["Glucose"].value_counts()
```



	count
Glucose	
99	17
100	17
111	14
125	14
129	14
...	...
56	1
169	1
149	1
65	1
190	1

136 rows × 1 columns

dtype: int64

```
data.head()
```



	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1



Next steps:

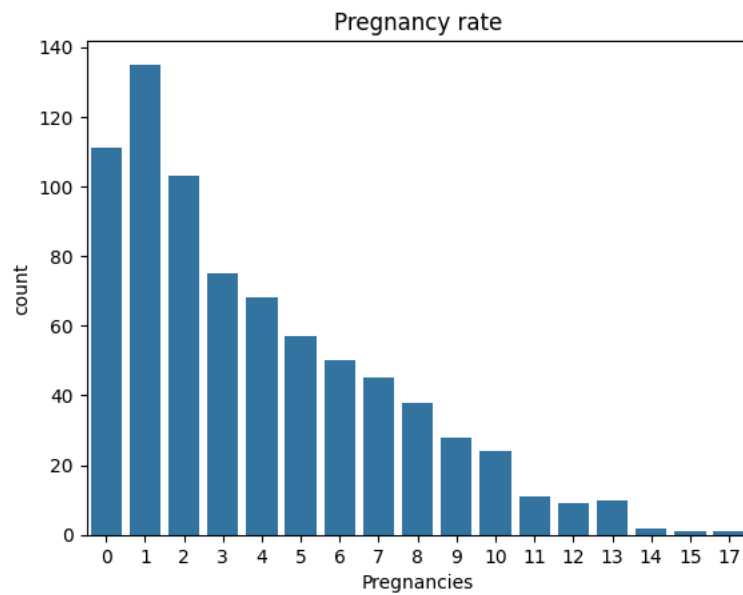
[Generate code with data](#)

☒ [View recommended plots](#)

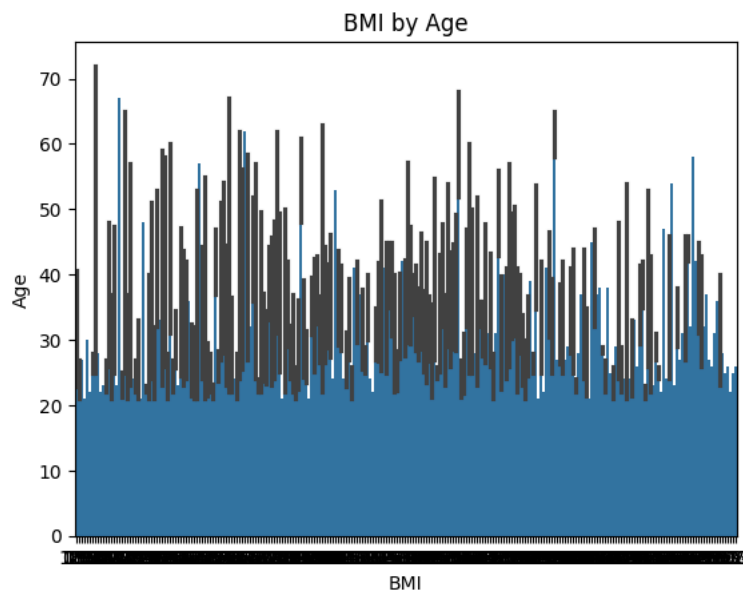
[New interactive sheet](#)

VISUALIZATION

```
sns.countplot(x="Pregnancies",data=data)
plt.title("Pregnancy rate")
plt.show()
```



```
sns.barplot(x="BMI",y="Age",data=data)
plt.title("BMI by Age")
plt.show()
```

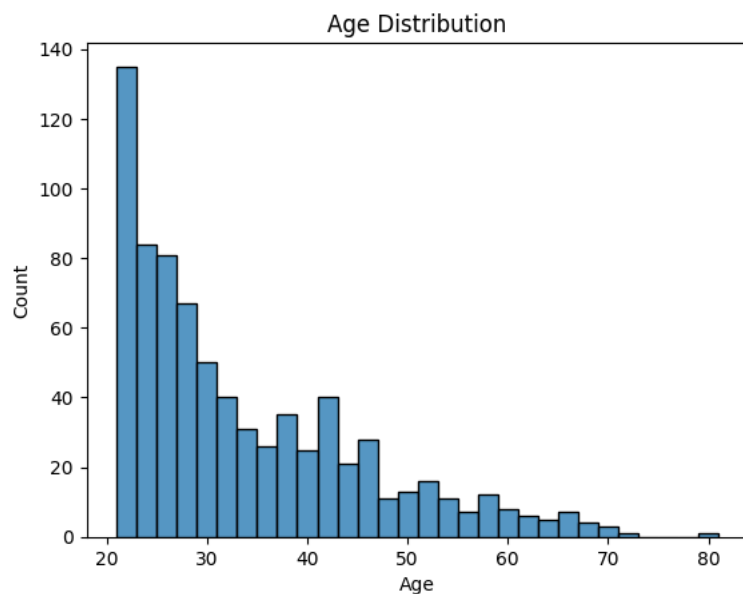


```
pregnancy_by_age=data.groupby("Age")["Pregnancies"].value_counts(normalize=True).unstack()
print("Pregnancy by Age :")
print(pregnancy_by_age)
```



40	NaN	NaN	NaN	NaN	NaN
41	0.045455	0.045455	NaN	NaN	NaN
42	NaN	0.055556	NaN	NaN	NaN
43	NaN	0.076923	NaN	0.076923	NaN
44	0.250000	0.250000	NaN	NaN	NaN
45	NaN	0.066667	NaN	NaN	NaN
46	0.153846	NaN	0.076923	NaN	NaN
47	NaN	NaN	NaN	NaN	0.166667
48	0.200000	NaN	NaN	NaN	NaN
49	NaN	NaN	NaN	NaN	NaN
50	NaN	NaN	NaN	NaN	NaN
51	NaN	NaN	NaN	NaN	NaN
52	NaN	0.125000	NaN	NaN	NaN
53	NaN	NaN	NaN	NaN	NaN
54	NaN	NaN	NaN	NaN	NaN
55	NaN	NaN	NaN	NaN	NaN
56	NaN	NaN	NaN	NaN	NaN
57	NaN	0.200000	NaN	NaN	NaN
58	0.142857	NaN	NaN	NaN	NaN
59	NaN	NaN	NaN	NaN	NaN
60	NaN	NaN	NaN	NaN	NaN
61	NaN	NaN	NaN	NaN	NaN
62	0.250000	NaN	NaN	NaN	NaN
63	NaN	NaN	NaN	NaN	NaN
64	NaN	NaN	NaN	NaN	NaN
65	NaN	NaN	NaN	NaN	NaN
66	NaN	NaN	NaN	NaN	NaN
67	NaN	NaN	NaN	NaN	NaN
68	NaN	NaN	NaN	NaN	NaN
69	NaN	NaN	NaN	NaN	NaN
70	NaN	NaN	NaN	NaN	NaN
72	NaN	NaN	NaN	NaN	NaN
81	NaN	NaN	NaN	NaN	NaN

```
sns.histplot(data["Age"],bins=30)
plt.title("Age Distribution")
plt.show()
```

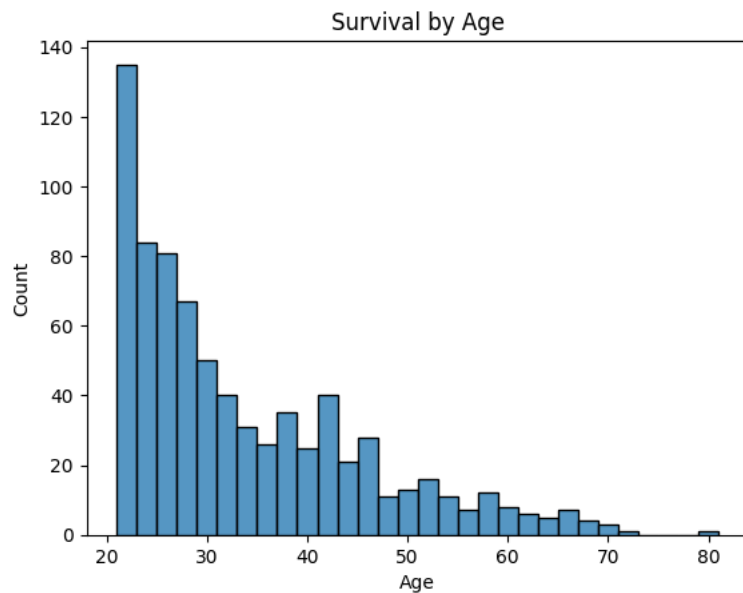


```
data["Age"].median()
```



29.0

```
sns.histplot(data=data, x='Age', multiple='stack', bins=30)
plt.title('Survival by Age')
plt.show()
```



Feature Scaling

```
data.head()
```



	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Next steps:

[Generate code with data](#)
[View recommended plots](#)
[New interactive sheet](#)

```
data.head()
```



	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Next steps:

[Generate code with data](#)
[View recommended plots](#)
[New interactive sheet](#)

```
x=data.drop("Outcome",axis=1)
y=data["Outcome"]
```

```
print(x.head())
print(y.head())
```



```

Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin  BMI  \
0            6      148           72           35         0  33.6
1            1       85           66           29         0  26.6
2            8      183           64            0         0  23.3
3            1       89           66           23        94  28.1
4            0      137           40           35       168  43.1

DiabetesPedigreeFunction  Age
0                0.627     50
1                0.351     31
2                0.672     32
3                0.167     21
4                2.288     33
0            1
1            0
```

```

2    1
3    0
4    1
Name: Outcome, dtype: int64

```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```

print(len(x_train))
print(len(y_train))

```

```

↗ 614
614

```

```
from sklearn.preprocessing import StandardScaler
```

```

scaler=StandardScaler()
x_train_scaled=scaler.fit_transform(x_train)
x_test_scaled=scaler.fit_transform(x_test)
#Convert back to DataFrame for better readability(optional):
x_train_scaled=pd.DataFrame(x_train_scaled,columns=x_train.columns,index=x_train.index)
x_test_scaled=pd.DataFrame(x_test_scaled,columns=x_test.columns,index=x_test.index)

```

```

print("Scaled Training Data:")
display(x_train_scaled.head())

```

```

print("\nScaled test Data")
display(x_test_scaled.head())

```

```

↗ Scaled Training Data:

```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
60	-0.526397	-1.151398	-3.752683	-1.322774	-0.701206	-4.135256	-0.490735	-1.035940
618	1.588046	-0.276643	0.680345	0.233505	-0.701206	-0.489169	2.415030	1.487101
346	-0.828460	0.566871	-1.265862	-0.090720	0.013448	-0.424522	0.549161	-0.948939
294	-1.130523	1.254179	-1.049617	-1.322774	-0.701206	-1.303720	-0.639291	2.792122
231	0.681856	0.410665	0.572222	1.076490	2.484601	1.838121	-0.686829	1.139095

Scaled test Data

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
668	0.491242	-0.726678	-0.441476	0.667308	1.051919	0.233748	-0.170941	0.666612
324	-0.633683	-0.285349	0.318908	0.611305	-0.660221	0.435379	-1.086080	-1.073358
624	-0.633683	-0.411443	-0.173105	-1.180790	-0.660221	-0.145794	-1.053629	-1.073358
690	1.053704	-0.442966	0.542551	-1.180790	-0.660221	-0.881156	1.211504	-0.045194
473	0.772473	0.471215	0.989836	-1.180790	-0.660221	-0.252540	-0.884879	1.220238

```

print("Training Data Shape",x_train_scaled.shape)
print("Test Data Shape",x_test_scaled.shape)

```

```

↗ Training Data Shape (614, 8)
Test Data Shape (154, 8)

```

LOGISTIC REGRESSION

```

model_log=LogisticRegression(max_iter=200)
model_log.fit(x_train_scaled,y_train)

```

```

↗ LogisticRegression
LogisticRegression(max_iter=200)

```

```

pred_log=model_log.predict(x_test_scaled)
print(pred_log)

```

```

↗ [0 0 0 0 0 0 0 1 0 1 0 1 0 0 0 0 0 0 1 1 0 0 0 0 0 1 0 0 0 0 1 1 1 1 1 1 1
  0 0 1 0 1 1 0 0 0 1 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 0 0 0 1 0 1 0 1 1 0 0 0
  0 0 0 0 0 0 1 0 0 0 0 1 1 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 1 0 1 0 0 1 0 1 0
  0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 1 1 1 1 1 0 0 1 0 0 1 1 0 0 0 0 0 0 0 0 0
  0 1 0 0 0 0]

```

```
print("Accuracy:",accuracy_score(y_test,pred_log))
print(classification_report(y_test,pred_log))
```

```
Accuracy: 0.7857142857142857
```

	precision	recall	f1-score	support
0	0.81	0.87	0.84	99
1	0.73	0.64	0.68	55
accuracy			0.79	154
macro avg	0.77	0.75	0.76	154
weighted avg	0.78	0.79	0.78	154

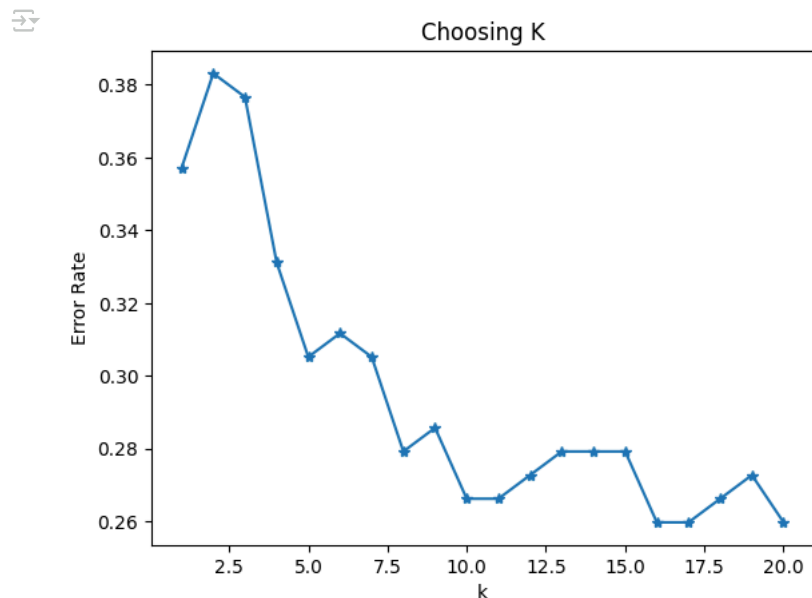
```
model_knn=KNeighborsClassifier(n_neighbors=7)
model_knn.fit(x_train_scaled,y_train)
pred_knn=model_knn.predict(x_test_scaled)
```

```
print("Accuracy:",accuracy_score(y_test,pred_knn))
print(classification_report(y_test,pred_knn))
```

```
Accuracy: 0.6948051948051948
```

	precision	recall	f1-score	support
0	0.75	0.79	0.77	99
1	0.58	0.53	0.55	55
accuracy			0.69	154
macro avg	0.67	0.66	0.66	154
weighted avg	0.69	0.69	0.69	154

```
errors=[]
for k in range(1,21):
    knn=KNeighborsClassifier(n_neighbors=k)
    knn.fit(x_train_scaled,y_train)
    pred_k=knn.predict(x_test_scaled)
    errors.append(1-accuracy_score(y_test,pred_k))
plt.plot(range(1,21),errors,marker="*")
plt.xlabel("k")
plt.ylabel("Error Rate")
plt.title("Choosing K")
plt.show()
```



```
model_tree=DecisionTreeClassifier(max_depth=4)
model_tree.fit(x_train_scaled,y_train)
pred_tree=model_tree.predict(x_test_scaled)
```

```
print("Accuracy:",accuracy_score(y_test,pred_tree))
print(classification_report(y_test,pred_tree))
```

```
plt.figure(figsize=(12,8))
plot_tree(model_tree,filled=True,feature_names=x.columns)
plt.show()
```



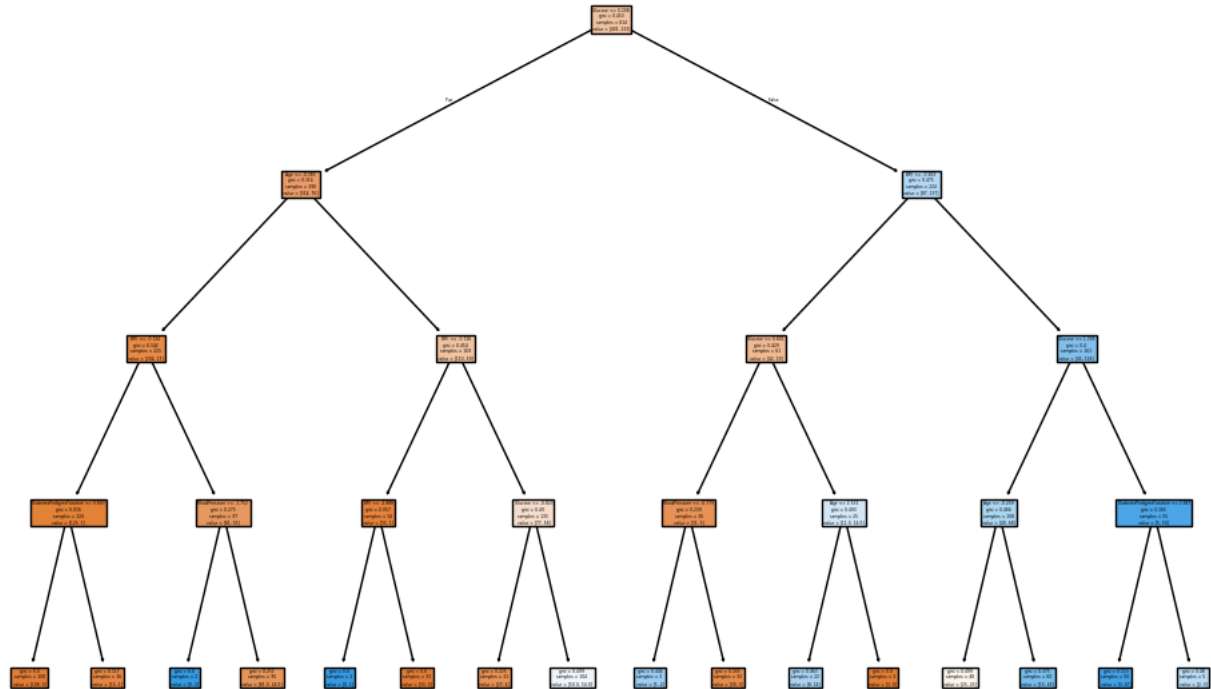
```

Accuracy: 0.6818181818181818
precision    recall  f1-score   support

      0       0.78    0.70    0.74      99
      1       0.55    0.65    0.60      55

 accuracy          0.68      154
  macro avg       0.66    0.68    0.67      154
 weighted avg     0.70    0.68    0.69      154

```



```

model_rf=RandomForestClassifier(n_estimators=100,random_state=42)
model_rf.fit(x_train_scaled,y_train)
pred_rf=model_rf.predict(x_test_scaled)

print("Accuracy:",accuracy_score(y_test,pred_rf))
print(classification_report(y_test,pred_rf))

```

```

Accuracy: 0.7597402597402597
precision    recall  f1-score   support

      0       0.79    0.85    0.82      99
      1       0.69    0.60    0.64      55

 accuracy          0.76      154
  macro avg       0.74    0.72    0.73      154
 weighted avg     0.75    0.76    0.76      154

```

```

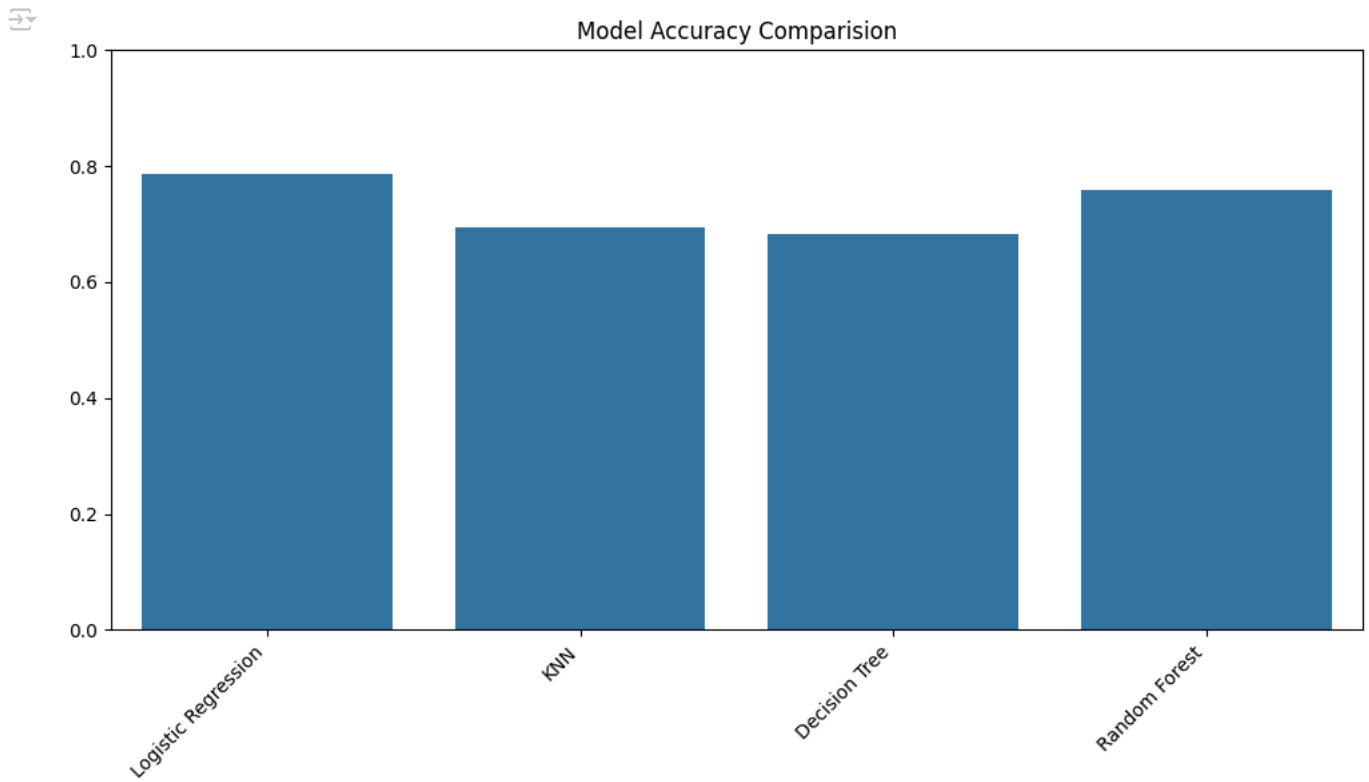
models=["Logistic Regression","KNN","Decision Tree","Random Forest"]
accuracies=[accuracy_score(y_test,pred_log),
            accuracy_score(y_test, pred_knn),
            accuracy_score(y_test, pred_tree),
            accuracy_score(y_test, pred_rf)
            ]

```

```

plt.figure(figsize=(10,6))
sns.barplot(x=models,y=accuracies)
plt.title("Model Accuracy Comparision")
plt.ylim(0,1)
plt.xticks(rotation=45,ha="right")
plt.tight_layout()
plt.show()

```



```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'KNN': KNeighborsClassifier(),
    'Random Forest': RandomForestClassifier(),
    'Decision Trees': DecisionTreeClassifier()
}
```

```
results = {}
```

```
for name, model in models.items():
    model.fit(x_train_scaled, y_train)
    y_pred = model.predict(x_test_scaled)
    cm = confusion_matrix(y_test, y_pred)
    print(f"\n Confusion Matrix for {name}:")
    print(cm)
    print(classification_report(y_test, y_pred))
    results[name] = (model, y_pred)
```



Confusion Matrix for Logistic Regression:

```
[[86 13]
 [20 35]]
```

	precision	recall	f1-score	support
0	0.81	0.87	0.84	99
1	0.73	0.64	0.68	55
accuracy			0.79	154
macro avg	0.77	0.75	0.76	154
weighted avg	0.78	0.79	0.78	154

Confusion Matrix for KNN:

```
[[83 16]
 [31 24]]
```

	precision	recall	f1-score	support
0	0.73	0.84	0.78	99
1	0.60	0.44	0.51	55
accuracy			0.69	154
macro avg	0.66	0.64	0.64	154
weighted avg	0.68	0.69	0.68	154

Confusion Matrix for Random Forest:

```
[[83 16]
 [24 31]]
```

	precision	recall	f1-score	support
0	0.78	0.84	0.81	99
1	0.66	0.56	0.61	55

accuracy			0.74	154
macro avg	0.72	0.70	0.71	154
weighted avg	0.73	0.74	0.74	154

Confusion Matrix for Decision Trees:

[[76 23]				
[17 38]]				
	precision	recall	f1-score	support
0	0.82	0.77	0.79	99
1	0.62	0.69	0.66	55

accuracy			0.74	154
macro avg	0.72	0.73	0.72	154
weighted avg	0.75	0.74	0.74	154

```
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, roc_auc_score

plt.figure(figsize=(10,6))
for name ,(model, _) in results .items():
    y_prob=model.predict_proba(x_test_scaled)[: ,1]
    fpr,tpr, _=roc_curve(y_test,y_prob)
    auc_score=roc_auc_score(y_test,y_prob)
    plt.plot(fpr, tpr, label=f"{name} (AUC = {auc_score:.2f})")

plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve Comparison')
plt.legend()
plt.grid()
plt.show()
```

